Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 9

Aim

Implement a Recurrent Neural Network using Long Short-Term Memory (LSTM) cells on the IMDB movie review dataset for binary sentiment classification, and evaluate model performance with and without hyperparameter tuning.

---

1. Dataset Source

- Dataset: IMDB Movie Reviews Sentiment Dataset
- Built into Keras: keras.datasets.imdb — no external download required
- Original source: Andrew L. Maas et al., Stanford AI Lab, 2011
- Reference: “Learning Word Vectors for Sentiment Analysis”, ACL 2011

---

2. Dataset Description

The IMDB dataset is a standard benchmark for natural language processing and sentiment analysis tasks. It contains 50,000 movie reviews sourced from the Internet Movie Database (IMDB), each labeled as either positive or negative based on user star ratings.

Key properties:

- Total size: 50,000 reviews — 25,000 training, 25,000 test (pre-split)
- Target variable: Sentiment (0 = Negative, 1 = Positive) — perfectly balanced (12,500 per class)
- Text representation: Reviews are pre-tokenized and encoded as sequences of integer word indices
- Vocabulary: Top 10,000 most frequent words retained; less frequent words are discarded
- Sequence length: Variable (reviews range from tens to hundreds of words); padded/truncated to a fixed length of 200 tokens
- Preprocessing applied: pad_sequences with padding=‘post’, truncating=‘post’ to enforce uniform input shape (25000, 200)

---

3. Mathematical Formulation of the Algorithm

An LSTM (Long Short-Term Memory) is a specialized type of Recurrent Neural Network (RNN) designed to learn long-range dependencies in sequential data. Standard RNNs suffer from the vanishing gradient problem over long sequences; LSTMs address this with a gated cell state that can carry information across many time steps.

3.1 Word Embedding

Each input word index w_t is first mapped to a dense continuous vector via a learned embedding matrix $E \in \mathbb{R}^{V \times d}$:

$$x_t = E[w_t] \in \mathbb{R}^d$$

where V = 10,000 (vocabulary size) and d is the embedding dimension (128 in the baseline). The embedding layer learns to encode semantic similarity between words during training.

3.2 LSTM Gates and Cell State

At each time step t, the LSTM receives the current input x_t and the previous hidden state h_{t-1}. Four gating operations control the flow of information:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \leftarrow \text{Forget gate}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \leftarrow \text{Input gate}$$

$$g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \quad \leftarrow \text{Candidate cell values}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \leftarrow \text{Output gate}$$

where σ is the sigmoid function, W are learned weight matrices, b are bias terms, and $[h_{t-1}, x_t]$ denotes concatenation of the previous hidden state and current input.

3.3 Cell State and Hidden State Updates

The cell state C_t acts as a long-term memory conveyor, updated by selectively forgetting old information and adding new candidate values:

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t$$

The hidden state (short-term memory output) is computed as:

$$h_t = o_t \odot \tanh(C_t)$$

where $\odot$ denotes element-wise multiplication. After processing all 200 tokens, the final hidden state h_T is used as the sequence representation.

3.4 Binary Classification Output

The final hidden state h_T is passed to a Dense(1) layer with Sigmoid activation to produce a probability estimate for the positive sentiment class:

$$\hat{y} = \sigma(W_{out} \cdot h_T + b_{out})$$

The predicted class is: 1 (Positive) if $\hat{y} > 0.5$, else 0 (Negative).

3.5 Loss Function and Optimizer

Binary cross-entropy loss for m training samples:

$$J = -(1/m)\,\Sigma_i\,[\,y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\,]$$

Weights are updated via the Adam optimizer using adaptive moment estimates (same formulation as in Experiments 7 and 8). Gradients are propagated back through time (BPTT) through all 200 time steps of the LSTM.

---

4. Algorithm Limitations

1. Sequential Computation (No Parallelism)

LSTM computation is inherently sequential — each time step depends on the previous hidden state, so the 200 steps cannot be parallelized across the sequence dimension. This makes LSTMs significantly slower to train than Transformer-based models (e.g., BERT) which process all positions simultaneously.

2. Difficulty with Very Long Sequences

Although LSTMs mitigate the vanishing gradient problem via the cell state, they still struggle to reliably retain information across very long sequences (hundreds to thousands of tokens). For reviews much longer than 200 tokens, relevant context from the beginning may be diluted by the end.

3. Fixed Vocabulary (Out-of-Vocabulary Problem)

The model is trained on the top 10,000 most frequent words. Any word not in this vocabulary is silently discarded. Rare words, proper nouns, or domain-specific terminology that carry sentiment information are lost, limiting the model's ability to generalize to unseen text.

4. Hyperparameter Sensitivity

LSTM performance is sensitive to embedding dimension, number of LSTM units, dropout rate, learning rate, and sequence length. Poor combinations — especially a high learning rate with a small LSTM — can cause fast but unstable convergence, as observed in this experiment where the tuner's selected lr=0.01 led to early stopping at epoch 4.

5. No Pre-Trained Language Knowledge

The embedding layer is trained from scratch on only 25,000 reviews. Pre-trained word vectors (GloVe, Word2Vec) or contextual embeddings (BERT, GPT) would provide richer semantic representations, particularly for infrequent or domain-specific words that the current model cannot distinguish.

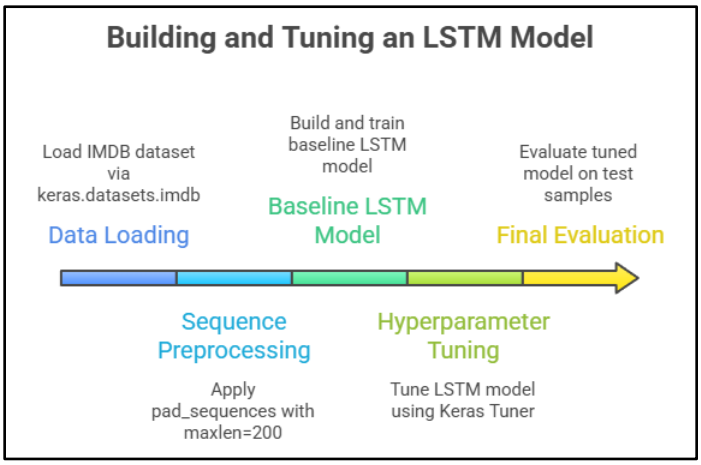
6. Computationally Expensive Relative to Performance

Each LSTM unit maintains four gate matrices (forget, input, candidate, output), making the parameter count and training cost roughly 4× that of a comparable simple RNN. For many NLP tasks, simpler models (logistic regression on TF-IDF features) or modern attention-based models achieve competitive or superior accuracy at lower computational cost.

---

5. Methodology / Workflow

1. **Data Loading**
   - Load IMDB dataset via keras.datasets.imdb.load_data(num_words=10000) — returns pre-split integer-encoded sequences.
   - Confirm split: 25,000 training samples, 25,000 test samples; perfectly balanced (12,500 per class).
2. **Sequence Preprocessing**
   - Apply pad_sequences with maxlen=200, padding='post', truncating='post' to enforce uniform shape (N, 200).
   - Reviews shorter than 200 words are zero-padded; reviews longer than 200 words are truncated from the end.
3. **Baseline LSTM Model (Without Tuning)**
   - Build: Embedding(10000, 128) → LSTM(64) → Dense(1, Sigmoid). Total: 1,329,473 parameters.
   - Compile with Adam(lr=0.001) and binary_crossentropy. Train for 5 epochs, batch size 64, 10% validation split.
   - Evaluate: accuracy, classification report (precision, recall, F1 for both classes), confusion matrix, training curves.
4. **Hyperparameter Tuning (Keras Tuner RandomSearch)**
   - Tune: embed_dim ∈ {64, 128}, lstm_units ∈ {32, 64, 128}, dropout ∈ [0.2, 0.4], learning_rate ∈ {1e–2, 1e–3, 1e–4}.
   - Search on 5,000-sample subset (3 epochs per trial, 10 trials). Objective: maximize val_accuracy.
   - Retrain best model on full 25,000 training samples for up to 10 epochs with EarlyStopping(patience=2, restore_best_weights=True).
5. **Final Evaluation and Comparison**
   - Evaluate tuned model on the same 25,000 test samples. Compare accuracy, per-class metrics, and confusion matrix against the baseline.
   - Analyse and document any divergence between tuning-subset performance and full-dataset performance.



6. Performance Analysis

Architecture: Embedding(10000, 128) → LSTM(64) → Dense(1, Sigmoid)

Total Parameters: 1,329,473 | Optimizer: Adam(lr=0.001) | Epochs: 5 | Batch Size: 64

Train/Test Split: 25,000 / 25,000 | Classes: Negative (0), Positive (1) — perfectly balanced

Metric	Negative	Positive	Overall
Precision	0.85	0.85	—
Recall	0.85	0.85	—
F1-Score	0.85	0.85	—
Support	12500	12500	25000
Accuracy	—	—	84.81%

1. Symmetric Class Performance

Both Negative and Positive classes achieve identical precision, recall, and F1-score of 0.85. This symmetry reflects the perfectly balanced class distribution (12,500 per class) — the LSTM has equal exposure to both classes and has learned equally strong representations for both positive and negative sentiment patterns.

2. LSTM Captures Sequential Sentiment

An 84.81% accuracy from a single-layer LSTM with only 5 training epochs demonstrates that the architecture successfully captures sequential dependencies in review text. The LSTM's gated cell state allows it to retain relevant sentiment-bearing words (e.g., 'excellent', 'terrible', 'disappointing') across the 200-token sequence while suppressing irrelevant filler content.

3. Baseline Confusion Matrix

With 84.81% accuracy on 25,000 test samples, approximately 21,203 reviews are correctly classified and 3,797 are misclassified. The equal error rate across both classes (approximately 1,899 errors per class) confirms the model is not biased toward either sentiment.

4. Overall

The baseline LSTM — a 128-dimensional embedding feeding a 64-unit LSTM with Adam(lr=0.001) — is a strong, untuned starting point for sentiment analysis. The 84.81% baseline accuracy is well above chance (50%) and competitive with

classical approaches (logistic regression on TF-IDF typically achieves 85–87%), achieved without any manual feature engineering.

---

7. Hyperparameter Tuning

Before Tuning — Baseline Architecture

Hyperparameter	Baseline Value
embed_dim	128
lstm_units	64
dropout	None
learning_rate	0.001
epochs	5
batch_size	64

Baseline Test Accuracy: 84.81% | Macro F1: 0.85

Keras Tuner — RandomSearch Configuration:

- embed_dim: Choice of [64, 128]
- lstm_units: Choice of [32, 64, 128]
- dropout: Float in [0.2, 0.4] (step 0.1)
- learning_rate: Choice of [1e–2, 1e–3, 1e–4]
- max_trials = 10, objective = 'val_accuracy', seed = 42
- Search subset: 5,000 samples | Search epochs: 3 per trial
- Final training: up to 10 epochs with EarlyStopping(patience=2, monitor='val_accuracy', restore_best_weights=True)

Best Hyperparameters Found:

Hyperparameter	Baseline Value	Best Value
embed_dim	128	128
lstm_units	64	32
dropout	None	0.3
learning_rate	0.001	0.01
Epochs (stopped at)	5	4

After Tuning — Results:

Metric	Negative	Positive	Overall

Precision	0.81	0.85	—
Recall	0.86	0.80	—
F1-Score	0.83	0.82	—
Support	12500	12500	25000
Accuracy	—	—	82.98%

Key Observations:

- Accuracy decreased from 84.81% → 82.98% (−1.83 percentage points) — the tuned configuration underperformed the baseline on the full test set
- Root cause — search subset mismatch: the tuner searched on only 5,000 samples (20% of training data); a configuration optimal for a small subset may not generalize to the full 25,000-sample training distribution, particularly for an NLP task where vocabulary coverage matters
- High learning rate (0.01) with a small LSTM (32 units) caused rapid but unstable convergence — early stopping triggered at epoch 4, before the model fully learned the sentiment boundary, whereas the baseline trained stably for all 5 epochs at lr=0.001
- Reducing LSTM units (64 → 32) halves the model's sequential memory capacity — a 32-unit LSTM has limited ability to simultaneously track multiple sentiment-relevant features across a 200-token review
- Dropout of 0.3 on LSTM recurrent connections is appropriate regularisation, but its benefit is negated by the aggressive learning rate and reduced model capacity
- Negative recall improved (0.85 → 0.86) while Positive recall fell (0.85 → 0.80) — the tuned model becomes slightly biased toward predicting Negative, increasing false negatives for Positive reviews
- This result demonstrates a key limitation of tuning on a small subset for sequence models: LSTM performance on text data is sensitive to the full vocabulary distribution, which a 5,000-sample subset cannot adequately represent

---

8. Code and Output – Without Tuning

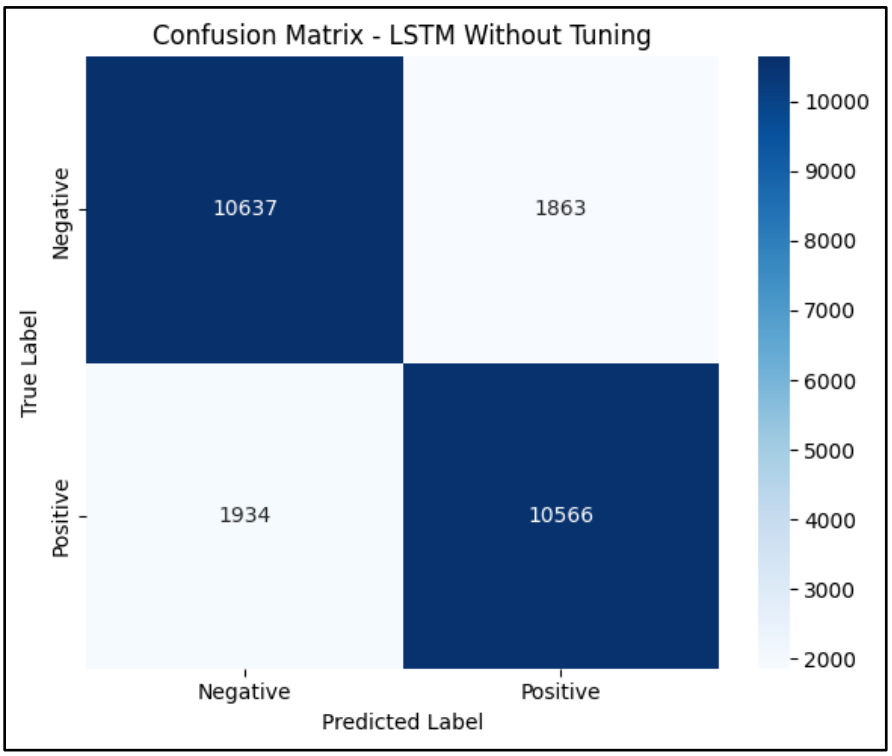
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from
tensorflow.keras.preprocessing.sequenc
e import pad_sequences
from sklearn.metrics import
classification_report, confusion_matrix
import seaborn as sns

NUM_WORDS = 10000   # vocabulary
size
MAXLEN    = 200     # max review
length (words)

# Load IMDB
```

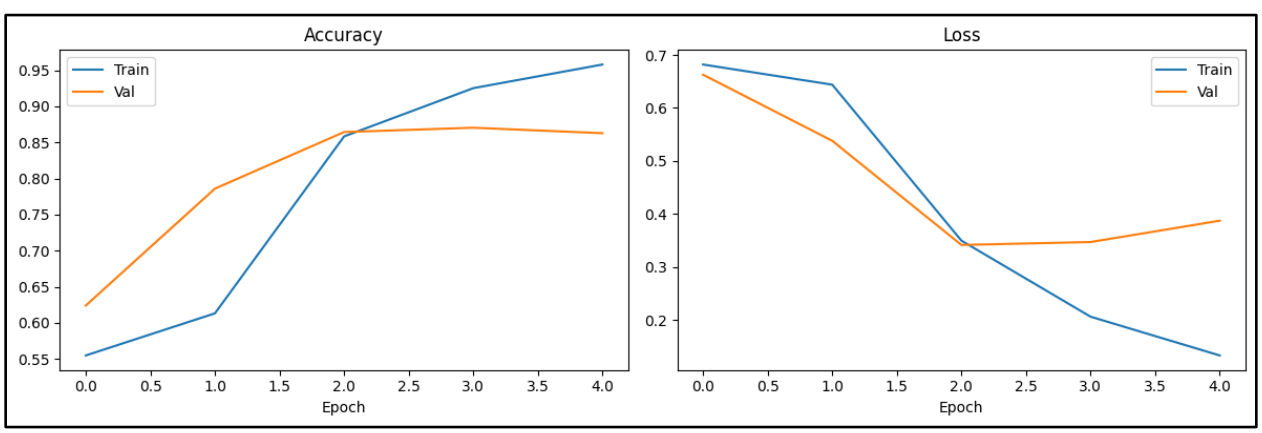
```
(X_train, y_train), (X_test, y_test) =
keras.datasets.imdb.load_data(num_wo
rds=NUM_WORDS)

print(f"Training samples: {len(X_train)},
Test samples: {len(X_test)}")
print(f"Classes:
{np.unique(y_train)}  (0=Negative,
1=Positive)")

# Pad/truncate sequences to uniform
length
X_train = pad_sequences(X_train,
maxlen=MAXLEN, padding='post',
truncating='post')
X_test  =
pad_sequences(X_test,  maxlen=MAXL
EN, padding='post', truncating='post')

print(f"Input shape after padding:
{X_train.shape}")

# Build LSTM model
model = keras.Sequential([
    layers.Embedding(NUM_WORDS,
128, input_length=MAXLEN),
    layers.LSTM(64),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer=keras.optimize
rs.Adam(learning_rate=0.001),
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train
history = model.fit(X_train, y_train,
epochs=5, batch_size=64,
                    validation_split=0.1,
verbose=1)

# Evaluate
test_loss, test_acc =
model.evaluate(X_test, y_test,
verbose=0)
print(f"\nTest Accuracy:
{test_acc*100:.2f}%")

y_pred = (model.predict(X_test) >
0.5).astype(int).flatten()

print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=["Negative",
"Positive"]))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
            xticklabels=["Negative",
"Positive"],
            yticklabels=["Negative",
"Positive"])
plt.title('Confusion Matrix - LSTM
Without Tuning')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()

# Training curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Train')
plt.plot(history.history['val_accuracy'],
label='Val')
plt.title('Accuracy'); plt.xlabel('Epoch');
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],
label='Train')
plt.plot(history.history['val_loss'],
label='Val')
```

```
plt.title('Loss'); plt.xlabel('Epoch');
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
Epoch 1/5
352/352 ━━━━━━━━━━━━━━━━━━━━ 94s 260ms/step - accuracy: 0.5548 - loss: 0.6819 - val_accuracy: 0.6240 - val_loss: 0.6627
Epoch 2/5
352/352 ━━━━━━━━━━━━━━━━━━━━ 89s 252ms/step - accuracy: 0.6132 - loss: 0.6439 - val_accuracy: 0.7860 - val_loss: 0.5381
Epoch 3/5
352/352 ━━━━━━━━━━━━━━━━━━━━ 141s 250ms/step - accuracy: 0.8584 - loss: 0.3498 - val_accuracy: 0.8644 - val_loss: 0.3417
Epoch 4/5
352/352 ━━━━━━━━━━━━━━━━━━━━ 90s 255ms/step - accuracy: 0.9252 - loss: 0.2064 - val_accuracy: 0.8704 - val_loss: 0.3472
Epoch 5/5
352/352 ━━━━━━━━━━━━━━━━━━━━ 90s 254ms/step - accuracy: 0.9580 - loss: 0.1333 - val_accuracy: 0.8628 - val_loss: 0.3874

Test Accuracy: 84.81%
782/782 ━━━━━━━━━━━━━━━━━━━━ 27s 35ms/step
```

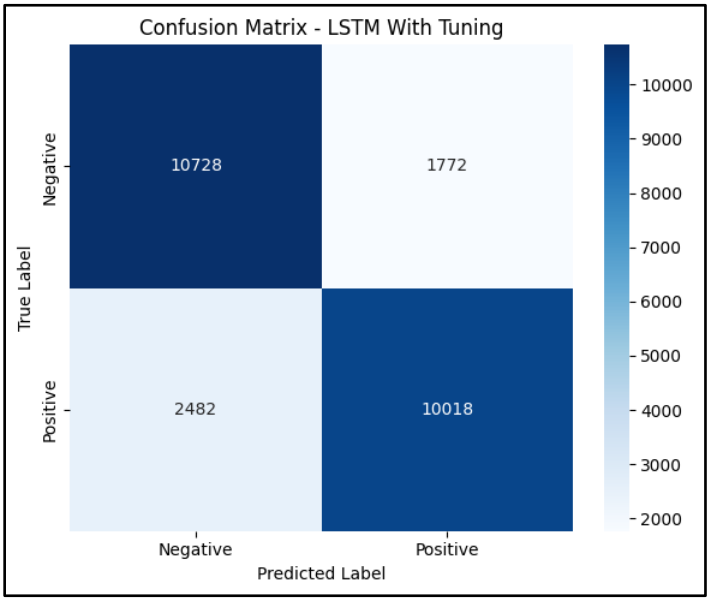




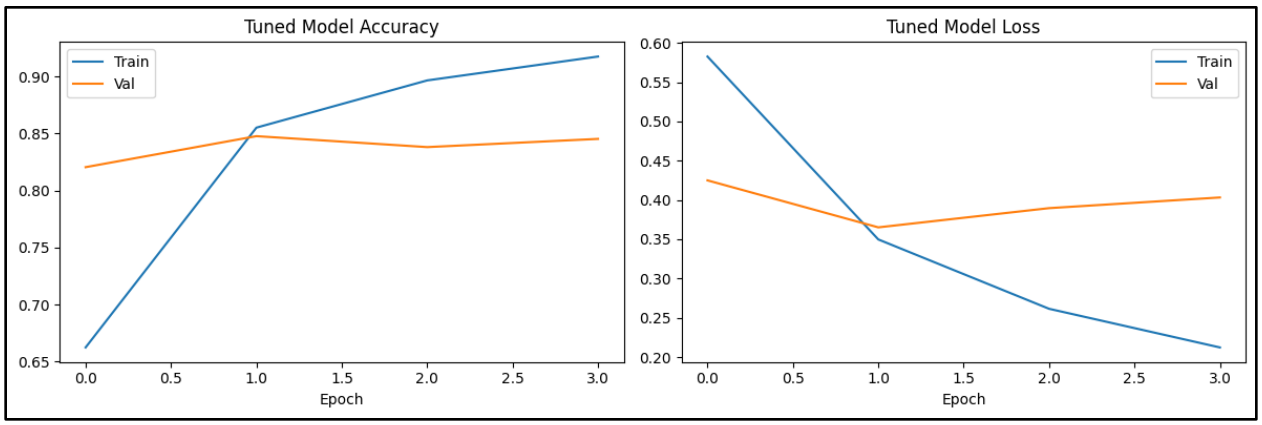
9. Code and Output – With Tuning

```
!pip install keras-tuner -q

import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
from
tensorflow.keras.preprocessing.sequenc
e import pad_sequences
import keras_tuner as kt
from sklearn.metrics import
classification_report, confusion_matrix
import seaborn as sns

NUM_WORDS = 10000
MAXLEN    = 200

(X_train, y_train), (X_test, y_test) =
keras.datasets.imdb.load_data(num_wo
rds=NUM_WORDS)
X_train = pad_sequences(X_train,
maxlen=MAXLEN, padding='post',
truncating='post')
X_test  =
pad_sequences(X_test,  maxlen=MAXL
EN, padding='post', truncating='post')

# Subset for faster search
X_tune = X_train[:5000]
y_tune = y_train[:5000]

def build_model(hp):
    embed_dim  =
hp.Choice('embed_dim',  [64, 128])
    lstm_units = hp.Choice('lstm_units',
[32, 64, 128])
    dropout    = hp.Float('dropout', 0.2,
0.4, step=0.1)
    lr         = hp.Choice('learning_rate',
[1e-2, 1e-3, 1e-4])

    model = keras.Sequential([
        layers.Embedding(NUM_WORDS,
embed_dim, input_length=MAXLEN),
        layers.LSTM(lstm_units,
dropout=dropout),
        layers.Dense(1,
activation='sigmoid')
    ])
    model.compile(optimizer=keras.optimi
zers.Adam(learning_rate=lr),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=10,
    seed=42,
    directory='lstm_tuning',
    project_name='imdb_lstm'
)

print("Starting hyperparameter search
(10 trials)...")
tuner.search(X_tune, y_tune, epochs=3,
validation_split=0.2, verbose=0)

best_hps =
tuner.get_best_hyperparameters(1)[0]
print("\nBest Hyperparameters:")
print(f"  embed_dim     :
{best_hps.get('embed_dim')}")
print(f"  lstm_units    :
{best_hps.get('lstm_units')}")
print(f"  dropout       :
{best_hps.get('dropout')}")
print(f"  learning_rate :
{best_hps.get('learning_rate')}")

# Train best model on full data
best_model =
tuner.hypermodel.build(best_hps)
early_stop =
keras.callbacks.EarlyStopping(monitor='
val_accuracy', patience=2,
                              restore_best
_weights=True)
```

```
history = best_model.fit(X_train, y_train,
epochs=10, batch_size=64,
                    validation_split=0.1,
callbacks=[early_stop], verbose=1)

print(f"\nStopped at epoch:
{len(history.history['accuracy'])}")

test_loss, test_acc =
best_model.evaluate(X_test, y_test,
verbose=0)
print(f"Test Accuracy:
{test_acc*100:.2f}%")

y_pred = (best_model.predict(X_test) >
0.5).astype(int).flatten()

print("\nClassification Report:")
print(classification_report(y_test,
y_pred, target_names=["Negative",
"Positive"]))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues',
          xticklabels=["Negative",
"Positive"],
          yticklabels=["Negative",
"Positive"])
plt.title('Confusion Matrix - LSTM With
Tuning')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()

# Training curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Train')
plt.plot(history.history['val_accuracy'],
label='Val')
plt.title('Tuned Model Accuracy');
plt.xlabel('Epoch'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],
label='Train')
plt.plot(history.history['val_loss'],
label='Val')
plt.title('Tuned Model Loss');
plt.xlabel('Epoch'); plt.legend()
plt.tight_layout()
plt.show()
```



```
Best Hyperparameters:
  embed_dim     : 128
  lstm_units    : 32
  dropout       : 0.30000000000000004
  learning_rate : 0.01
Epoch 1/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 53s 142ms/step - accuracy: 0.6623 - loss: 0.5827 - val_accuracy: 0.8204 - val_loss: 0.4250
Epoch 2/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 50s 141ms/step - accuracy: 0.8551 - loss: 0.3497 - val_accuracy: 0.8476 - val_loss: 0.3651
Epoch 3/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 48s 137ms/step - accuracy: 0.8965 - loss: 0.2614 - val_accuracy: 0.8380 - val_loss: 0.3896
Epoch 4/10
352/352 ━━━━━━━━━━━━━━━━━━━━ 50s 141ms/step - accuracy: 0.9174 - loss: 0.2122 - val_accuracy: 0.8452 - val_loss: 0.4032

Stopped at epoch: 4
```



10. Conclusion

This experiment demonstrates the application of Long Short-Term Memory networks for natural language processing through binary sentiment classification on the IMDB movie review dataset. The baseline LSTM — Embedding(128) → LSTM(64) → Sigmoid — achieves 84.81% accuracy in just 5 epochs on 25,000 test reviews with perfectly symmetric performance across both sentiment classes (F1=0.85). The gated architecture of the LSTM, with its explicit forget, input, and output gates, enables the network to selectively retain sentiment-bearing words across variable-length sequences while filtering out irrelevant filler content, without requiring any manual feature engineering or preprocessing beyond tokenization and padding.

The hyperparameter tuning phase yields an important practical insight: tuning on a limited subset (5,000 samples) identified lr=0.01 with lstm_units=32 as the best configuration, which led to fast but unstable convergence (stopping at epoch 4) and a reduced accuracy of 82.98% on the full test set. This outcome illustrates a critical principle in LSTM tuning — the search subset must adequately represent the full training vocabulary distribution, and aggressive learning rates can negate the benefits of smaller, more regularised architectures. The baseline configuration (lr=0.001, 64 units, no dropout) proves more robust for sequence classification on large, lexically diverse datasets.